

S9 — Memoria del Sistema

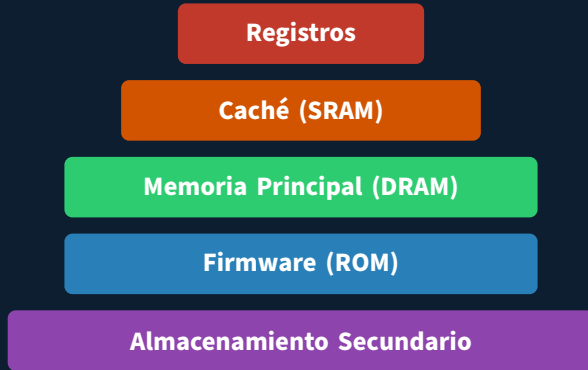
Lenin G. Falconí

Indicaciones

- Reparto del tema por grupos:
 - G3: Estructura del sistema de Memoria + Memoria Caché.
 - G4: Memoria Interna (DRAM, SRAM, ROM), Corrección de Errores (Hamming), Organización Avanzada.
- Fuentes bibliográficas:
 - **stallings2006esp**<empty citation>, 7ma edición, 2006, Capítulos 5 y 6.
 - **stallings2022computer**<empty citation>, 11ava edición, 2022, Capítulos 4, 5 y 6.

Estructura del sistema de Memoria

Jerarquía de Memoria



Hacia abajo: ↑ capacidad, ↓ costo/bit | Hacia arriba: ↑ velocidad

Memoria Interna

DRAM y SRAM: Principios Básicos

DRAM (RAM Dinámica)

El estándar para Memoria Principal.

X

- **Estructura:** 1 transistor + 1 condensador.
- **Lógica:** El bit se almacena como *carga eléctrica* (naturaleza analógica).
- **Reto:** El condensador pierde carga rápidamente. Requiere un **refresco periódico** para no perder los datos.

SRAM (RAM Estática)

El estándar para Memoria Caché.

- **Estructura:** 6 transistores configurados como un biestable (Flip-Flop).
- **Lógica:** Dispositivo puramente digital.
- **Ventaja:** Mientras haya energía, el dato es estable. **No necesita refrescos.**

DRAM vs SRAM: La gran decisión

¿Por qué no usar la memoria más rápida en todo el sistema? La respuesta está en la física:

	DRAM (Dinámica)	SRAM (Estática)
Densidad (Costo)	Muy Alta (Barata)	Baja (Costosa)
Velocidad	Lenta	Muy Rápida
Refresco	Sí (Consume ciclos)	No (Constante)
Uso Ideal	Memoria Principal (RAM)	Memoria Caché (L1/L2)

Evolución de las Memorias ROM

La necesidad de flexibilidad

La ROM nació como "Solo Lectura", pero la necesidad de actualizar el firmware impulsó métodos de borrado cada vez más avanzados.

- **ROM (Máscara):** Programada físicamente en fábrica. **Imborrable** y cero margen de error.

Evolución de las Memorias ROM

La necesidad de flexibilidad

La ROM nació como "Solo Lectura", pero la necesidad de actualizar el firmware impulsó métodos de borrado cada vez más avanzados.

- **ROM (Máscara):** Programada físicamente en fábrica. **Imborrable** y cero margen de error.
- **PROM:** Programable por el usuario *una sola vez* (quemando fusibles internos).

Evolución de las Memorias ROM

La necesidad de flexibilidad

La ROM nació como "Solo Lectura", pero la necesidad de actualizar el firmware impulsó métodos de borrado cada vez más avanzados.

- **ROM (Máscara):** Programada físicamente en fábrica. **Imborrable** y cero margen de error.
- **PROM:** Programable por el usuario *una sola vez* (quemando fusibles internos).
- **EPROM:** Borrable con luz ultravioleta (tarda hasta 20 minutos y borra *todo* el chip).

Evolución de las Memorias ROM

La necesidad de flexibilidad

La ROM nació como "Solo Lectura", pero la necesidad de actualizar el firmware impulsó métodos de borrado cada vez más avanzados.

- **ROM (Máscara):** Programada físicamente en fábrica. **Imborrable** y cero margen de error.
- **PROM:** Programable por el usuario *una sola vez* (quemando fusibles internos).
- **EPROM:** Borrable con luz ultravioleta (tarda hasta 20 minutos y borra *todo* el chip).
- **EEPROM:** Borrable eléctricamente a nivel de byte. Flexible, pero costosa y menos densa.

Evolución de las Memorias ROM

La necesidad de flexibilidad

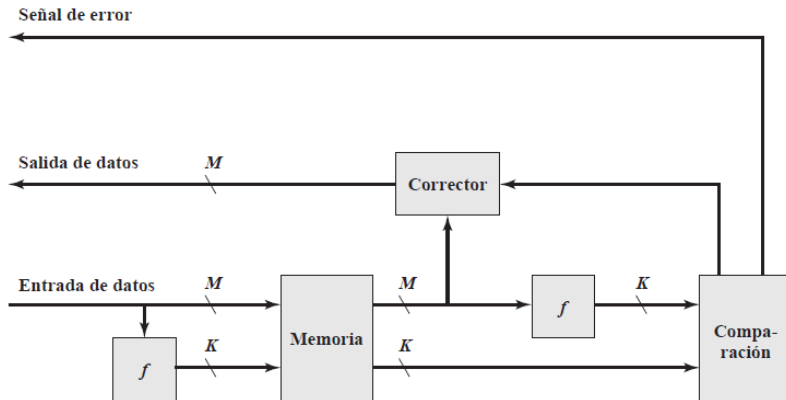
La ROM nació como "Solo Lectura", pero la necesidad de actualizar el firmware impulsó métodos de borrado cada vez más avanzados.

- **ROM (Máscara):** Programada físicamente en fábrica. **Imborrable** y cero margen de error.
- **PROM:** Programable por el usuario *una sola vez* (quemando fusibles internos).
- **EPROM:** Borrable con luz ultravioleta (tarda hasta 20 minutos y borra *todo* el chip).
- **EEPROM:** Borrable eléctricamente a nivel de byte. Flexible, pero costosa y menos densa.
- **Flash Memory:** Borrable eléctricamente **por bloques**. Combina la altísima densidad de EPROM con la flexibilidad eléctrica. ¡El estándar actual!

Corrección de Errores

Control de Errores en Memoria

Para lidiar con los errores, se implementan lógicas de detección y corrección en el flujo de lectura y escritura de la memoria.



Hard Error vs Soft Error

Los sistemas de memoria están sujetos a fallos. Se dividen en dos familias principales:

Hard Error (Fallo Físico)

Un defecto **permanente** en el hardware. Celdas fundidas que se quedan atascadas en 0 o 1.

→ **Causas:** Desgaste, defectos de fábrica.

→ **Solución:** **Irreversible**. Hay que reemplazar el chip.

Soft Error (Fallo Transitorio)

Un evento **aleatorio y no destructivo**. Un bit "voltea" de 0 a 1 por accidente, pero el chip está sano.

→ **Causas:** Ruido eléctrico, **radiación natural** (partículas alfa).

→ **Solución:** **Corregible**. Si lo detectamos, podemos volver a escribir el bit correcto.

Código de Hamming: Concepto General

Para corregir un *Soft Error*, no basta con saber *que* ocurrió un error, tenemos que saber **exactamente dónde ocurrió**.

- **La idea:** Añadir bits redundantes (paridad) a nuestros datos.
- Richard Hamming descubrió que ubicando estos guardianes en posiciones que son **potencias de 2**, podemos generar un código matemático infalible.
- El cruce de información (intersección en los diagramas de Venn) delata matemáticamente al bit dañado.

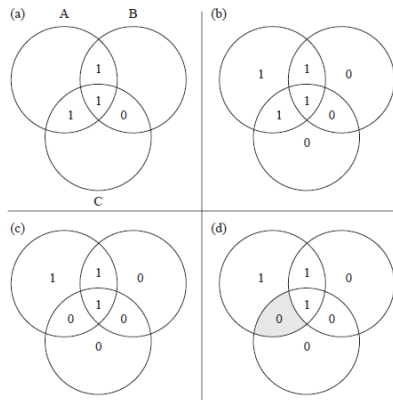


Figura 5.8. Código corrector de errores de Hamming.

Código de Hamming: Generación (Paso 1)

- 1. Datos a enviar (4 bits):** 1011 (D1, D2, D3, D4)
- 2. Posiciones:** Metemos los datos dejando libres las potencias de 2 (P1, P2, P4).



3. Cálculo de la Paridad (Las zonas vigiladas deben sumar una cantidad par de '1's):

- **P1** vigila saltando de 1 en 1 (pos 1, 3, 5, 7): $P1 \oplus 1 \oplus 0 \oplus 1 = 0 \implies \mathbf{P1 = 0}$
- **P2** vigila saltando de 2 en 2 (pos 2, 3, 6, 7): $P2 \oplus 1 \oplus 1 \oplus 1 = 0 \implies \mathbf{P2 = 1}$
- **P4** vigila saltando de 4 en 4 (pos 4, 5, 6, 7): $P4 \oplus 0 \oplus 1 \oplus 1 = 0 \implies \mathbf{P4 = 0}$

Mensaje final transmitido hacia la memoria:



Código de Hamming: Error y Corrección (Paso 2)

¡Impacto de partícula radiactiva!

El bit en la **posición 6** cambia de 1 a 0 de forma silenciosa.

Palabra dañada leída por el procesador:

1	2	3	4	5	6	7
0	1	1	0	0	0	1

4. Detección (Síndrome): El hardware recalcula las sumas de paridad.

- **S1** (pos 1, 3, 5, 7): $0 \oplus 1 \oplus 0 \oplus 1 = 0$ (¡Todo bien!)
- **S2** (pos 2, 3, 6, 7): $1 \oplus 1 \oplus 0 \oplus 1 = 1$ (¡Hay un error!)
- **S4** (pos 4, 5, 6, 7): $0 \oplus 0 \oplus 0 \oplus 1 = 1$ (¡Hay un error!)

La magia matemática

Tomamos los valores de error y armamos el Síndrome binario ($S4\ S2\ S1$) = $110_2 = 6$.

Organización Avanzada de Memorias RAM

- Desarrollo del resto del tema por parte del grupo de exposición.